

PROJECT STRUCTURE

```
.gitignore
CLAUDE.md
README.md
app/build.gradle.kts
app/src/commonMain/kotlin/app/followlens/ui/App.kt
build.gradle.kts
core-diff/build.gradle.kts
core-diff/src/commonMain/kotlin/app/followlens/diff/ExportParser.kt
core-diff/src/commonMain/kotlin/app/followlens/diff/FollowDiff.kt
core-diff/src/commonMain/kotlin/app/followlens/diff/Models.kt
core-diff/src/commonTest/kotlin/app/followlens/diff/FollowDiffTest.kt
data/build.gradle.kts
data/src/commonMain/sqldelight/app/followlens/data/FollowLens.sq
docs/PROJECT_PLAN.md
gradle.properties
gradle/libs.versions.toml
settings.gradle.kts
```

Plan: "Non-Follower" Instagram Analyzer App – v1 Offline, Scalable to a Social Media Manager

Context

The user wants to build and launch (App Store + Play Store) a professional app that:

- ****v1 (100% offline):**** the user downloads their own data export from Instagram; the app parses the **followers** and **following** lists and shows the ****list of usernames the user follows that do not follow them back****. No network, no login, no account.
- ****Future (online, subscription SaaS):**** a social-media-manager product – follower-change monitoring, content planning + scheduled publishing, and a unified message/comment inbox.

Decisions locked in

Question	Decision
Mobile framework	**Kotlin Multiplatform + Compose Multiplatform** (one language end-to-end; the diff engine is shared code reused by the future Spring Boot backend)
Follow/unfollow automation	**Not automated – ever.** v1 shows the non-follower list from the user's export. Future = an <i>*assisted*</i> review queue (app ranks, one tap opens the profile, the human confirms). Automating follow/unfollow bans accounts and gets the app pulled; the official API has no such endpoint.
Other future automation	Content scheduling + publishing and the inbox **are** allowed – only via the official Instagram Graph API for Business/Creator accounts (rate-limited; DMs only within Meta's 24h window).
v1 scope	**Core + history:** non-follower list, fans list, whitelist, snapshot history (gained/lost over time across imports). Single account.
Monetization	**Fully free for v1.** No IAP SDK, no ads – keeps the app genuinely offline and the privacy story clean. Monetize when the online product launches.

hes. |
| Brand / folder | ****FollowLens****, new repo at `~/Projects/follow-lens` – fully
separate from the current `~/Desktop/Instagram App/Insta Follow Check` folder, w
hich the user will delete. |

Two facts that shaped the plan

1. ****Spring Boot cannot be shipped to the stores.**** It is a server-side JVM fram
ework. The
store artifact is a Kotlin Multiplatform mobile app. Spring Boot is reserved
for the future
backend.
2. ****The current repo is throwaway scaffolding**** – a Spring Initializr project w
ith ~90
dependencies (every starter checked) and only the default `InstaFollowCheckAp
plication` stub.
It gets restructured (see "Repo restructure" below).

Architecture

v1 – on-device only

...

Kotlin Multiplatform app (iOS + Android, one codebase)

- └ :core-diff pure Kotlin module – parsing + set math, no dependencies, he
avily unit-tested
- └ :data SQLite via SQLDelight – snapshot history + whitelist
- └ :app Compose Multiplatform UI – onboarding, import, dashboard, li
sts, history, settings
- └ NO backend, NO auth, NO network permission, NO analytics/ads SDK

...

Future – add the backend without rewriting

...

KMP app —HTTPS→ Spring Boot backend (Kotlin)

- └ depends on the SAME :core-diff module
- └ Auth: Instagram Login / OAuth2 (Spring Security)
- └ Snapshot sync + trend history
- └ @Scheduled / Quartz – periodic follower-count + insigh
ts pull
- └ Official IG Graph API client – Content Publishing, Mes
saging, Insights
- └ Billing: RevenueCat webhooks → entitlements
- └ PostgreSQL

...

`:core-diff` is the seam – the identical parsing/diff code runs on-device now an
d server-side later.

Core feature (v1)

Instagram data export

User path: Instagram → Settings → Accounts Center → Your information and permiss
ions → Download
your information → "Some of your information" → ****Followers and following**** → fo
rmat ****JSON**** →

link arrives by email (minutes to ~a day - onboarding copy must set this expectation).

ZIP contains `connections/followers\_and\_following/`:

- `followers\_1.json` (`\_2`, ... when large) - **top-level JSON array**
- `following.json` - object, key `relationships\_following` → array
- also present, useful later: `pending\_follow\_requests.json`, `recently\_unfollowed\_profiles.json`,
`blocked\_profiles.json`, `close\_friends.json`

Entry shape:

```
```json
{ "string_list_data": [
 { "href": "https://www.instagram.com/USERNAME", "value": "USERNAME", "timestamp": 1700000000 }
] }
```
```

`:core-diff` module (the heart of the app)

Pure Kotlin. Input: `followers: Set<Account>`, `following: Set<Account>` (Account = username + firstSeen timestamp). Outputs:

- **notFollowingBack** = following - followers ← the headline list
- **fans** = followers - following
- **mutuals** = followers ∩ following
- with a prior snapshot: `newFollowers`, `lostFollowers`, `newlyFollowed`, `youUnfollowed`
- whitelist applied as the final filter

Tests must cover: multi-part `followers\_N.json`, username case-folding, deactivated accounts (empty `value`), older **HTML** export fallback, duplicates, empty/corrupt file, wrong file picked.

Storage - SQLDelight (SQLite)

...

```
snapshot(id, taken_at, source_export_date)
account(id, username)
snapshot_account(snapshot_id, account_id, role)    -- FOLLOWER | FOLLOWING
whitelist(account_id, added_at)
```

Same rows the future backend will sync.

Screens

1. **Onboarding / How to export** - steps with screenshots; "we never see your password, nothing leaves your phone"
2. **Import** - file picker (`.zip` or the two `.json`s), parse progress, clear validation errors
3. **Dashboard** - followers / following / non-follower counts, mini trend chart, last import date
4. **Non-followers** - the list; per row: username, first-seen date, actions: *Open in Instagram*
(`instagram://user?username=`), *Whitelist*, *Copy*
5. **Fans** - they follow you, you don't follow back
6. **History / Changes** - since last import: gained / lost / new non-followers
7. **Settings** - manage whitelist, clear all data, export CSV, privacy policy

Store-compliance checklist (before feature code)

- ****Name & icon:**** no "Insta"/"Gram"/"IG", no Instagram glyph or gradient. Pick a neutral brand (e.g. **FollowLens**, **Circleback**, **Orbit**). Meta enforces this; stores reject on it. Choose the name now – it flows into bundle IDs and store assets.
- ****App Store 5.2.1 / 4.2:**** position as **analytics on the user's own exported data**. Never request Instagram credentials; no login automation or scraping.
- ****Privacy:**** ship a privacy-policy URL (required by both stores even offline). Fill Apple's Privacy Nutrition Label and Play's Data Safety form as ****"no data collected"***** – true here.
- ****Play:**** target current API level; no ``QUERY_ALL_PACKAGES``. No network permission in v1.

Graphics / design direction

- ****Design tool:**** Figma (free). One screen kit + a Compose-mappable component set.
- ****Visual language:**** data-forward and calm – a tool, not a toy. Neutral surface, one accent color (not Instagram purple/pink), generous whitespace, tabular-aligned numbers in lists, friendly empty-states.
- ****Charts:**** follower trend line + gained/lost bars – Compose libs: Koala Plot or ``compose-charts``, Material 3 theming.
- ****Motion:**** a short Lottie clip on onboarding (request export → import → results).
- ****Icon:**** an abstract mark (orbit / venn / arrows), not a camera.
- ****Store assets:**** icon (all sizes), 5–8 screenshots per platform/device size, Play feature graphic, optional 15–30s preview video.

Future roadmap (compliant framing)

| Stated goal | Compliant version | Mechanism |
|---|---|---|
| Check follows/unfollows every 24h | Remind the user to re-export; if they connect a Business account, auto-pull ****counts + insights**** and show deltas | Spring ``@Scheduled`` / Quartz + IG Graph API (counts/insights only) |
| Auto-unfollow non-followers | ****Assisted unfollow**** – ranked queue, one tap opens the profile, user confirms; optional N/day cap | Deep links only; no automated action |
| Add pics / write blogs | ****Content planner + scheduled publish**** (images/video / reels/stories/carousels) | IG ****Content Publishing API****, Business/Creator, rate-limited |
| Reply to messages | ****Unified inbox**** for Business accounts inside Meta's messaging window | IG ****Messaging API**** (24h window) |
| Subscription SaaS | Tiered subscription | RevenueCat + backend entitlement check |

Anything needing the user's password, the private mobile API, or performing follow/unfollow/like/comment on the user's behalf is ****permanently out of scope****.

New repo: `~/Projects/follow-lens`

The current `~/Desktop/Instagram App` folder is abandoned (user will delete it). Everything is created fresh in `~/Projects/follow-lens` as a multi-module Kotlin Multiplatform Gradle build:

```

```
follow-lens/
├─ settings.gradle.kts includes :core-diff, :data, :app (:backend added later)
├─ build.gradle.kts root – plugin versions via version catalog
├─ gradle/libs.versions.toml version catalog
├─ core-diff/ kotlin-multiplatform, only kotlinx-serialization-json
├─ data/ kotlin-multiplatform + SQLDelight
├─ app/ Compose Multiplatform
│ └─ src/commonMain shared UI
│ └─ androidApp Android entry point + manifest (NO INTERNET permission)
│ └─ iosApp iOS entry point (Xcode project)
├─ docs/
│ └─ PROJECT_PLAN.md the full plan (source for the PDF)
│ └─ FollowLens-Plan.pdf portable plan + structure, for handing to other AI / people
├─ CLAUDE.md auto-loaded by future Claude Code sessions
└─ README.md
```

- Nothing is carried over from the old Spring Boot scaffold. `backend/` (Spring Boot, Kotlin) is added only when the online phase starts, and will depend on `:core-diff`.
- Fastest bootstrap: JetBrains "Kotlin Multiplatform" project wizard (or `kmp.jetbrains.com` template), then add the `:core-diff` / `:data` modules.
- Tooling: Android Studio (or IntelliJ) + KMP plugin, JDK 21, Xcode for the iOS target.

## Session deliverables (immediately after approval)

1. Create `~/Projects/follow-lens/` with the module skeleton above (Gradle files, empty source dirs, `.gitignore`, `git init`).
2. `CLAUDE.md` – locked decisions, architecture, constraints, compliance rules, build order.
3. `docs/PROJECT\_PLAN.md` – this plan, copied in full.
4. `docs/FollowLens-Plan.pdf` – generated from the plan (via `pandoc`/Chrome headless/`textutil`, whichever is present) so it can be handed to another AI or a collaborator.
5. Persistent memory entries (framework, no-automation rule, v1 scope, free-for-now, folder path).

---

## Verification (v1)

1. **\*\*`:core-diff` unit tests\*\*** – fixture export files (real anonymized `follower\_s\_1.json` + `following.json`, plus multi-part, HTML, corrupt, empty); assert all four output sets and the

- snapshot-delta outputs. This is the correctness core – high coverage here.
2. **\*\*Import integration test\*\*** – feed a real `.zip``, assert counts against a hand-computed expected.
  3. **\*\*Manual E2E on device\*\*** – request a real export from a test IG account, import, spot-check the non-follower list against 5–10 accounts in the Instagram app.
  4. **\*\*Offline check\*\*** – airplane mode on; every v1 feature works. Confirm the build declares no network permission (Android) / no networking entitlement use (iOS).
  5. **\*\*Store pre-flight\*\*** – TestFlight + Play internal testing; complete both privacy forms; walk the 5.2.1 / naming review points before public submission.

---

## ## Suggested build order

1. Pick the brand name; restructure the repo into the KMP multi-module layout.
2. ``:core-diff`` – parser + diff engine + full test suite (no UI yet).
3. ``:data`` – SQLDelight schema + snapshot/whitelist repositories.
4. ``:app`` – import flow → non-follower list → dashboard → fans → history → settings.
5. Design pass in Figma in parallel with 2–4; apply Material 3 theme + charts.
6. Onboarding screens + copy; deep links; CSV export.
7. Store assets, privacy policy page, TestFlight / Play internal builds.
8. Public submission.